Tab 1

D1: Problem Formulation + Heuristics

Project 4 – Timetable Generator (L3)

Course: CS F407: Artificial Intelligence

Complexity Level: L3 (Large Instance with Soft Constraints)

---

1. Problem Description

1.1 Real-World Scenario

Academic timetable generation is a complex administrative problem faced by universities every semester. A department must schedule 30 or more courses into a limited number of rooms and timeslots while respecting instructor availability, room capacity, and pedagogical preferences. The challenge arises from balancing limited physical infrastructure (lecture halls, laboratories) with human constraints such as faculty availability and student workload distribution.

This project models the university scheduling process as a search problem. The objective is to construct a complete timetable where each course is assigned a room and a timeslot such that all hard constraints are satisfied and soft constraints are optimized.

1.2 Complexity Level: L3

We implement the **L3 (Large)** complexity level.

This involves:

- 30+ courses
- Multiple rooms with varying capacities
- Multiple discrete timeslots
- Strict enforcement of hard constraints:
    - No room double-booking
    - No instructor conflict
    - Room capacity ≥ course enrollment

Additionally, we incorporate **soft constraints**:

- Preferred time slots for instructors

- Minimization of gaps in daily teaching schedules
- Balanced daily course distribution

Thus, the problem becomes a **search-based optimization problem** with combinatorial growth in the state space.

1.3 Limitations of the Approach

Formulating timetable generation as classical state-space search leads to exponential state space growth. For |C| courses and |R||T| possible assignments per course, the upper bound is:

(|R| X |T|)^|C|

As |C| increases beyond 30, memory-intensive algorithms such as A* may struggle due to frontier size explosion. Specialized CSP solvers or local search algorithms (e.g., Min-Conflicts) may scale better in practice. However, this project deliberately models the problem within the classical search framework to analyze algorithmic properties rigorously.

---

2. Environment & PEAS Description

2.1 PEAS Framework

Performance Measure

- +100 for complete valid timetable
- $-\infty$ (failure) if any hard constraint is violated
- Soft constraint penalties:
  - −5 for scheduling outside preferred slot
  - −10 for daily load imbalance
  - −5 per unnecessary gap

Objective: Maximize performance score (equivalently minimize total cost).

Environment

- Set of 30+ courses
- Set of rooms with fixed capacities
- Set of discrete timeslots
- Instructor availability constraints
- Student enrollment sizes

This forms a finite scheduling matrix.

Actuators

- Assign(course c, room r, timeslot t)

Sensors

- Current timetable state
- Conflict detection function
- Instructor availability checker
- Room capacity verifier
- Unassigned course tracker

2.2 Environment Classification

Property	Classification	Justification
Observable	Fully Observable	The agent has complete knowledge of courses, rooms, constraints
Deterministic	Deterministic	Assigning a course deterministically changes the state
Episodic	Sequential	Each assignment affects future possibilities
Static	Static	Environment does not change during execution
Discrete	Discrete	Finite rooms, timeslots, courses
Single-Agent	Yes	Only one scheduling agent modifies the state

3. Search Formulation

Let:

- C = set of courses
- R = set of rooms
- T = set of timeslots

3.1 State Space (S)

A state s E S is a partial mapping:

s: C -> (R X T)

where:

C (subset) C

represents the subset of courses already scheduled.

Thus,

S = { s | s is a valid partial assignment}

3.2 Initial State

s_0 = emptyset

No courses are assigned.

3.3 Goal Test

Goal(s) = (|C'| = |C|) ^ (HardConstraintViolations = 0)

All courses must be assigned without violating hard constraints.

3.4 Actions

For state s:

A(s) = { Assign(c, r, t) | c E C \ C}

Preconditions

- Room r is free at time t
- Instructor of c is free at time t
- Room capacity ≥ enrollment

Effects

Result(s, Assign(c,r,t)) = s U {(c,r,t)}

3.5 Transition Model

Result: S X A -> S

Deterministic mapping of state and action to new state.

3.6 Path Cost Function

Each assignment incurs:

- Base cost = 1
- Soft constraint penalty = p

g(n) = g(parent) + 1 + p

Where:

p = soft constraint penalty incurred

3.7 State Space Size Analysis

If:

- |C| = 30
- |R| = r
- |T| = t

Each course has at most $|R| \times |T|$ assignment choices.

Upper bound:

$|S| <= (|R| X |T|)^{|C|}$

This demonstrates exponential growth in $|C|$.

---

4. Heuristic Design

We design two admissible heuristics.

Heuristic 1: Remaining Courses

Definition

$h_1(n) = |C| - |C'|$

(Number of unassigned courses)

Admissibility Proof

Each unassigned course requires at least one action.

Minimum cost per action = 1.

Thus true remaining cost $h^*(n) >= |C| - |C'|$.

$h_1(n) <= h^*(n)$

Therefore, h_1 is admissible.

Consistency

After assigning one course:

$h_1(n) = h_1(n') + 1$

Since step cost $c(n,a,n') \ge 1$:

$h_1(n) <= c(n,a,n') + h_1(n')$

Thus consistent.

Heuristic 2: Relaxed Penalty Estimate

Definition

$h_2(n) = h_1(n) + P(n)$

Where:

- $P(n)$ = minimum unavoidable soft constraint penalties
- Computed by relaxing room constraints but keeping instructor availability

---

Admissibility

By relaxing some constraints, we compute the *minimum possible future penalty*.

Actual solution must incur:

- At least one assignment cost per remaining course
- At least unavoidable penalty $P(n)$

Thus:

$h_2(n) <= h^*(n)$

Admissible.

Consistency

Assignment reduces remaining courses by 1.

Penalty cannot decrease more than cost of action.

$h_2(n) <= c(n,a,n') + h_2(n')$

Thus consistent.

Dominance

Since:

$P(n) >= 0$

$h_2(n) >= h_1(n)$

Therefore, h_2 dominates h_1.

A* with h_2 will expand fewer or equal nodes compared to h_1.

Tab 2

D2: Algorithm Analysis

Project: Timetable Generator (Project 4) **Complexity Level:** L3 (Large)

1. Algorithm Selection & Justification

For the L3 Timetable Generator, we have selected the following four algorithms to analyze and implement. These choices cover the spectrum of uninformed vs. informed search and memory-efficient vs. optimality-guaranteed approaches.

1. Depth-First Search (DFS) [Uninformed]

- **Justification:** DFS is the foundational algorithm for Constraint Satisfaction Problems (CSPs). Since our timetable problem has a fixed solution depth (every schedule must contain exactly $|C|$ courses), DFS avoids the pitfalls of infinite depth. Its linear space complexity $O(b \cdot m)$ is critical for our L3 complexity (30+ courses), where storing the full frontier might exhaust memory.

2. Uniform Cost Search (UCS) [Uninformed]

- **Justification:** Unlike DFS, UCS considers the path cost g(n). In our specific problem, g(n) represents the accumulation of **soft constraint penalties** (e.g., professor preferences, load balancing). UCS is essential to analyze because it guarantees the *optimal* schedule (lowest penalty score) purely based on accumulated cost, serving as a baseline for optimality.

3. Greedy Best-First Search [Informed]

- **Justification:** In large combinatorial spaces (L3), finding *any* valid solution is often prioritized over finding the *perfect* one. Greedy Search uses our heuristics (h2) to aggressively pursue the goal. We analyze this to demonstrate the trade-off between search speed and solution quality.

*4. A Search [Informed]**

- **Justification:** A* combines the optimality of UCS with the efficiency of heuristics. Given that we need to maximize our performance measure (+100 for success, minus penalties), A* is the theoretical gold standard. It allows us to validate the effectiveness of our admissible heuristics (h1 and h2) designed in D1.

---

2. Algorithm Analysis

Problem Variables:

- b: Effective branching factor (valid room/time pairs per course). Max $b \approx |R| \times |T|$.
- d: Depth of the shallowest solution. Since all $|C|$ courses must be scheduled, $d = |C|$.
- m: Maximum depth of the search tree. Also $m = |C|$.
- C^*: Cost of the optimal solution.
- ϵ: Minimum step cost (1+min_penalty).

Algorithm 1: Depth-First Search (DFS)

DFS explores the schedule by attempting to assign one course at a time, backtracking only when a hard constraint is violated.

A. Time Complexity DFS expands the deepest node first. In the worst case, it generates the entire tree.

- **Step-by-step Derivation:**
    1. At depth 0, we have b choices for the first course.
    2. At depth 1, we have b choices for the second course.
    3. This continues until depth $m = |C|$ (all courses assigned).
    4. Total nodes generated $\approx b0 + b1 + b2 + ... + bm$.
    5. This is a geometric series dominated by the last term bm.
- **Result:** O(bm)
- **Context:** With $|C| = 30$ and $b \approx 20$, 2030 is astronomical. However, hard constraints prune this effective bsignificantly in practice.

B. Space Complexity DFS only needs to store the current path from the root to the leaf and the unexpanded siblings at each level.

- **Analysis:**
    - We store m nodes (the current partial schedule).
    - At each level k, we store $b-1$ unexpanded siblings.
    - Total storage $= O(b \cdot m)$.
- **Result:** $O(b \cdot |C|)$
- **Context:** This is **Linear Space**. For $|C| = 30$, this is negligible, making DFS the most memory-safe choice for our L3 complexity.

C. Completeness

- **Analysis:** DFS is complete in finite state spaces. Since our timetable has a finite number of courses, rooms, and timeslots, the graph is finite and contains no cycles (we never un-assign a course to progress).
- **Condition:** DFS will find a valid timetable if one exists, provided the search space is finite.

D. Optimality

- **Analysis:** DFS returns the *first* solution it finds (the leftmost valid path). It does not consider the path cost g(n) (soft constraint penalties).

- **Result: Not Optimal.** It might return a schedule with high penalties (e.g., unhappy professors) simply because it was found first.

Algorithm 2: Uniform Cost Search (UCS)

UCS expands the node with the lowest path cost $g(n)$, where $g(n)$ is the sum of assignment costs (1) plus soft constraint penalties (p).

A. Time Complexity UCS explores in contours of increasing cost.

- **Step-by-step Derivation:**
    1. Let $C*$ be the cost of the optimal solution.
    2. Let ϵ be the minimum cost of an action (base cost 1).
    3. The algorithm explores roughly up to depth $C*/\epsilon$.
    4. The number of nodes is roughly $bC*/\epsilon$.
- **Result:** $O(bC*/\epsilon)$
- **Context:** Since our depth is fixed at $|C|$ and every step costs at least 1, this effectively simplifies to $O(b|C|)$ in the worst case, similar to BFS.

B. Space Complexity UCS must keep the entire frontier of explored cost contours in memory to ensure no cheaper path exists.

- **Analysis:**
    - The frontier grows exponentially with the depth of the search.
    - It stores every node with cost $\leq C*$.
- **Result:** $O(bC*/\epsilon)$ or roughly $O(b|C|)$.
- **Context:** This is **Exponential Space**. For $|C|=30$, UCS will likely run out of memory (OOM) on a standard machine, highlighting the "Limitations" mentioned in D1.

C. Completeness

- **Analysis:** UCS is complete if the step cost $\epsilon>0$.
- **Proof:** Since every assignment adds at least 1 to the cost, the path cost $g(n)$ strictly increases. The algorithm will eventually reach the finite cost $C*$ and find the goal.

D. Optimality

- **Analysis:** UCS is guaranteed to be optimal.
- **Proof:** It expands nodes in strict increasing order of $g(n)$. The first goal node selected for expansion is guaranteed to have the minimum total cost (minimum soft constraint violations).

Algorithm 3: Greedy Best-First Search

Greedy Search expands the node that appears closest to the goal according to heuristic h(n) (e.g., h2: Relaxed Penalty Estimate). It ignores partial path cost g(n).

A. Time Complexity

- **Step-by-step Derivation:**
    1. In the worst case, the heuristic guides the search down a "false lead" (a path that looks good but hits a hard constraint dead-end).
    2. The agent must backtrack and try alternatives.
    3. In the worst case, it behaves like DFS.
- **Result:** $O(bm)=O(b|C|)$.
- **Context:** In the *best case*, the time complexity is O(m) (linear) if the heuristic is perfect and guides strictly to the goal.

B. Space Complexity

- **Analysis:** Greedy Search, like A* and BFS, keeps all generated nodes in memory to choose the best h(n).
- **Result:** $O(bm)=O(b|C|)$.
- **Context:** Unlike DFS, Greedy cannot discard siblings because a sibling might have a better heuristic value than a child of the current node. It suffers from exponential space complexity.

C. Completeness

- **Analysis:** Greedy search is **not complete** in infinite spaces (it can get stuck in loops).
- **Condition:** However, our space is **finite** and we do not reverse actions. Therefore, with repeated state checking (explored set), Greedy *is* complete for this specific problem, though it may visit all nodes in the worst case.

D. Optimality

- **Analysis:** Greedy search is **Not Optimal**.
- **Counterexample:** Suppose Path A incurs a massive penalty (p=100) in the first step but looks "close" to the finish (h=1). Path B incurs small penalties (p=5) but looks "far" (h=10). Greedy will choose Path A, locking in the high penalty, failing to find the globally optimal schedule.

Algorithm 4: A* Search

A* expands nodes based on $f(n)=g(n)+h(n)$, combining the actual penalty cost so far with the estimated remaining penalty. We use Heuristic h2 (Relaxed Penalty) from D1.

A. Time Complexity

- **Step-by-step Derivation:**
    1. Time complexity depends on the "error" of the heuristic $|h(n)-h*(n)|$.

    2. In the worst case (e.g., if h(n) is constant), A* expands all nodes with $f(n) < C*$.
- **Result:** $O(bd) = O(b^{|C|})$.
- **Context:** While exponential, the *effective* branching factor is significantly reduced by a good heuristic. With h2(admissible and consistent), A* prunes vast sections of the search tree that DFS would explore.

B. Space Complexity A* must store all generated nodes in the Open Set (Frontier) and Closed Set.

- **Analysis:**
    - The frontier grows as the surface area of the search contours.
- **Result:** $O(bd) = O(b^{|C|})$.
- **Context:** This is the primary bottleneck for A* in L3 projects. With 30 courses, the memory requirement is massive. (This validates why D1 mentioned CSP solvers might be preferred in real life, but A* gives us the theoretical optimum).

C. Completeness

- **Analysis:** A* is complete.
- **Proof:** As long as step costs are positive (which they are, min cost =1), A* will eventually traverse all nodes with $f(n) \leq C*$ and find the solution.

D. Optimality

- **Analysis:** A* is Optimal.
- **Proof:**
    1. We proved in D1 that h2 is **Admissible** ($h2 \leq h*$) and **Consistent**.
    2. Tree Search A* is optimal if h is admissible. Graph Search A* is optimal if h is consistent.
    3. Therefore, A* guarantees finding the schedule with the lowest possible combination of Base Costs + Soft Constraint Penalties.

D2: Algorithm Analysis

Project: Timetable Generator (Project 4)
Complexity Level: L3 (Large)

1. Algorithm Selection & Justification

For the L3 Timetable Generator, we have selected the following four algorithms to analyze and implement. These choices cover the spectrum of uninformed vs. informed search and memory-efficient vs. optimality-guaranteed approaches.

1. Depth-First Search (DFS) [Uninformed]

- **Justification:** DFS is the foundational algorithm for Constraint Satisfaction Problems (CSPs). Since our timetable problem has a fixed solution depth (every schedule must contain exactly $|C|$ courses), DFS avoids the pitfalls of infinite depth. Its linear space complexity $O(b \cdot m)$ is critical for our L3 complexity ($30+$ courses), where storing the full frontier might exhaust memory.

2. Uniform Cost Search (UCS) [Uninformed]

- **Justification:** Unlike DFS, UCS considers the path cost $g(n)$. In our specific problem, $g(n)$ represents the accumulation of **soft constraint penalties** (e.g., professor preferences, load balancing). UCS is essential to analyze because it guarantees the *optimal* schedule (lowest penalty score) purely based on accumulated cost, serving as a baseline for optimality.

3. Greedy Best-First Search [Informed]

- **Justification:** In large combinatorial spaces (L3), finding *any* valid solution is often prioritized over finding the *perfect* one. Greedy Search uses our heuristics ($h2$) to aggressively pursue the goal. We analyze this to demonstrate the trade-off between search speed and solution quality.

*4. A Search [Informed]**

- **Justification:** A* combines the optimality of UCS with the efficiency of heuristics. Given that we need to maximize our performance measure (+100 for success, minus penalties), A* is the theoretical gold standard. It allows us to validate the effectiveness of our admissible heuristics ($h1$ and $h2$) designed in D1.

2. Algorithm Analysis

Problem Variables:

- b: Effective branching factor (valid room/time pairs per course). Max $b \approx |R| \times |T|$.
- d: Depth of the shallowest solution. Since all $|C|$ courses must be scheduled, $d = |C|$.
- m: Maximum depth of the search tree. Also $m = |C|$.
- C^*: Cost of the optimal solution.
- ϵ: Minimum step cost ($1 + \text{min_penalty}$).

Algorithm 1: Depth-First Search (DFS)

DFS explores the schedule by attempting to assign one course at a time, backtracking only when a hard constraint is violated.

A. Time Complexity

DFS expands the deepest node first. In the worst case, it generates the entire tree.

- **Step-by-step Derivation:**
  1. At depth 0, we have b choices for the first course.
  2. At depth 1, we have b choices for the second course.
  3. This continues until depth $m = |C|$ (all courses assigned).
  4. Total nodes generated $\approx b^0 + b^1 + b^2 + ... + b^m$.
  5. This is a geometric series dominated by the last term b^m.
- **Result:** $O(b^m)$
- **Context:** With $|C| = 30$ and $b \approx 20$, 20^{30} is astronomical. However, hard constraints prune this effective b significantly in practice.

B. Space Complexity

DFS only needs to store the current path from the root to the leaf and the unexpanded siblings at each level.

- **Analysis:**
  - We store m nodes (the current partial schedule).
  - At each level k, we store $b - 1$ unexpanded siblings.
  - Total storage $= O(b \cdot m)$.
- **Result:** $O(b \cdot |C|)$
- **Context:** This is **Linear Space**. For $|C| = 30$, this is negligible, making DFS the most memory-safe choice for our L3 complexity.

C. Completeness

- **Analysis:** DFS is complete in finite state spaces. Since our timetable has a finite number of courses, rooms, and timeslots, the graph is finite and contains no cycles (we never un-assign a course to progress).
- **Condition:** DFS will find a valid timetable if one exists, provided the search space is finite.

D. Optimality

- **Analysis:** DFS returns the *first* solution it finds (the leftmost valid path). It does not consider the path cost $g(n)$ (soft constraint penalties).
- **Result: Not Optimal.** It might return a schedule with high penalties (e.g., unhappy professors) simply because it was found first.

Algorithm 2: Uniform Cost Search (UCS)

UCS expands the node with the lowest path cost $g(n)$, where $g(n)$ is the sum of assignment costs (1) plus soft constraint penalties (p).

A. Time Complexity

UCS explores in contours of increasing cost.

- **Step-by-step Derivation:**
  1. Let C^* be the cost of the optimal solution.
  2. Let ϵ be the minimum cost of an action (base cost 1).
  3. The algorithm explores roughly up to depth C^*/ϵ.
  4. The number of nodes is roughly $b^{C^*/\epsilon}$.
- **Result:** $O(b^{C^*/\epsilon})$
- **Context:** Since our depth is fixed at $|C|$ and every step costs at least 1, this effectively simplifies to $O(b^{|C|})$ in the worst case, similar to BFS.

B. Space Complexity

UCS must keep the entire frontier of explored cost contours in memory to ensure no cheaper path exists.

- **Analysis:**
  - The frontier grows exponentially with the depth of the search.
  - It stores every node with cost $\leq C^*$.
- **Result:** $O(b^{C^*/\epsilon})$ or roughly $O(b^{|C|})$.
- **Context:** This is **Exponential Space**. For $|C| = 30$, UCS will likely run out of memory (OOM) on a standard machine, highlighting the "Limitations" mentioned in D1.

C. Completeness

- **Analysis:** UCS is complete if the step cost $\epsilon > 0$.
- **Proof:** Since every assignment adds at least 1 to the cost, the path cost $g(n)$ strictly increases. The algorithm will eventually reach the finite cost C^* and find the goal.

D. Optimality

- **Analysis:** UCS is guaranteed to be optimal.
- **Proof:** It expands nodes in strict increasing order of $g(n)$. The first goal node selected for expansion is guaranteed to have the minimum total cost (minimum soft constraint violations).

Algorithm 3: Greedy Best-First Search

Greedy Search expands the node that appears closest to the goal according to heuristic $h(n)$ (e.g., $h2$: Relaxed Penalty Estimate). It ignores partial path cost $g(n)$.

A. Time Complexity

- **Step-by-step Derivation:**
  1. In the worst case, the heuristic guides the search down a "false lead" (a path that looks good but hits a hard constraint dead-end).
  2. The agent must backtrack and try alternatives.
  3. In the worst case, it behaves like DFS.
- **Result:** $O(b^m) = O(b^{|C|})$.
- **Context:** In the *best case*, the time complexity is $O(m)$ (linear) if the heuristic is perfect and guides strictly to the goal.

B. Space Complexity

- **Analysis:** Greedy Search, like A* and BFS, keeps all generated nodes in memory to choose the best $h(n)$.
- **Result:** $O(b^m) = O(b^{|C|})$.
- **Context:** Unlike DFS, Greedy cannot discard siblings because a sibling might have a better heuristic value than a child of the current node. It suffers from exponential space complexity.

C. Completeness

- **Analysis:** Greedy search is **not complete** in infinite spaces (it can get stuck in loops).
- **Condition:** However, our space is **finite** and we do not reverse actions. Therefore, with repeated state checking (explored set), Greedy *is* complete for this specific problem, though it may visit all nodes in the worst case.

D. Optimality

- **Analysis:** Greedy search is **Not Optimal**.
- **Counterexample:** Suppose Path A incurs a massive penalty ($p = 100$) in the first step but looks "close" to the finish ($h = 1$). Path B incurs small penalties ($p = 5$) but looks "far" ($h = 10$). Greedy will choose Path A, locking in the high penalty, failing to find the globally optimal schedule.

Algorithm 4: A* Search

A* expands nodes based on $f(n) = g(n) + h(n)$, combining the actual penalty cost so far with the estimated remaining penalty. We use Heuristic $h2$ (Relaxed Penalty) from D1.

A. Time Complexity

- **Step-by-step Derivation:**
  1. Time complexity depends on the "error" of the heuristic $|h(n) - h^*(n)|$.
  2. In the worst case (e.g., if $h(n)$ is constant), A* expands all nodes with $f(n) < C^*$.
- **Result:** $O(b^d) = O(b^{|C|})$.
- **Context:** While exponential, the *effective* branching factor is significantly reduced by a good heuristic. With $h2$ (admissible and consistent), A* prunes vast sections of the search tree that DFS would explore.

B. Space Complexity

A* must store all generated nodes in the Open Set (Frontier) and Closed Set.

- **Analysis:**
  - The frontier grows as the surface area of the search contours.
- **Result:** $O(b^d) = O(b^{|C|})$.
- **Context:** This is the primary bottleneck for A* in L3 projects. With 30 courses, the memory requirement is massive. (This validates why D1 mentioned CSP solvers might be preferred in real life, but A* gives us the theoretical optimum).

C. Completeness

- **Analysis:** A* is complete.
- **Proof:** As long as step costs are positive (which they are, min cost $= 1$), A* will eventually traverse all nodes with $f(n) \leq C^*$ and find the solution.

D. Optimality

- **Analysis:** A* is Optimal.
- **Proof:**
  1. We proved in D1 that $h2$ is **Admissible** ($h2 \leq h^*$) and **Consistent**.
  2. Tree Search A* is optimal if h is admissible. Graph Search A* is optimal if h is consistent.
  3. Therefore, A* guarantees finding the schedule with the lowest possible combination of Base Costs + Soft Constraint Penalties.

3. Summary Comparison Table

Feature	DFS	UCS	Greedy	A*
Time Complexity	$O(b^{\lvert C\rvert})$	$O(b^{\lvert C\rvert})$	$O(b^{\lvert C\rvert})$	$O(b^{\lvert C\rvert})$
Space Complexity	$O(b \cdot \lvert C\rvert)$	$O(b^{\lvert C\rvert})$	$O(b^{\lvert C\rvert})$	$O(b^{\lvert C\rvert})$
Completeness	Yes (finite)	Yes	Yes (finite)	Yes
Optimality	No	Yes	No	Yes
Memory Efficient	Yes	No	No	No
Cost-Aware	No	Yes	No	Yes